

세미나 발표자료 시나리오

2. AI 시대에 Git 활용법(with VSCode)
3. AI Coding과 Antigravity IDE 사용
4. 코덱스 플러그인 사용과 CLI 코딩

구성 기준: 각 주제별 HTML 슬라이드 발표자료 1개, 설명 약 40분 + 질의응답 약 10분

2번 발표 시나리오 - AI 시대에 Git 활용법(with VSCode)

발표 파일명: topic/github.html

발표 목표

이 발표는 Git 명령어를 모두 외우게 하는 것이 목적이 아니다. AI가 코드를 생성하거나 수정하는 시대에도, 개발자가 변경사항을 추적하고 검토하고 협업하기 위해 Git과 GitHub를 어떻게 활용해야 하는지 설명하는 것을 목표로 한다.

전체 시간 구성

구간	시간
도입: AI 시대에 Git이 왜 필요한가	5분
Git 기본 개념	8분
GitHub 협업 흐름	10분
VS Code에서 Git 사용하는 방법	10분
AI 도구와 Git을 연결하는 실무 흐름	5분
마무리	2분
합계	40분

Slide 1. 제목

화면 구성

- 제목: AI 시대에 Git 활용법(with VSCode)
- 부제: AI가 만든 코드를 안전하게 관리하는 방법
- 배경 이미지 후보: VS Code 화면 + GitHub 로고 + Git branch 아이콘

발표 내용

이번 발표에서는 AI 시대에 Git을 왜 계속 사용해야 하는지, 그리고 VS Code 환경에서 Git과 GitHub를 어떻게 활용할 수 있는지 설명하겠습니다.

AI 도구가 코드를 작성해주는 시대가 되었지만, AI가 만든 코드를 그대로 사용하는 것은 위험할 수 있습니다. 따라서 변경사항을 확인하고, 문제가 생겼을 때 되돌리고, 팀원에게 리뷰를 받는 과정이 필요합니다.

이때 가장 기본이 되는 도구가 Git과 GitHub입니다.

Slide 2. AI가 코드를 만들어도 Git이 필요한 이유

화면 구성

- 왼쪽: AI가 코드 생성
- 가운데: Git diff로 변경사항 확인
- 오른쪽: commit / PR / review
- 이미지 후보: AI -> Git -> Review 흐름도

발표 내용

AI 코딩 도구를 사용하면 HTML, C 코드, 문서, README 같은 파일을 빠르게 만들 수 있습니다.

하지만 AI가 항상 정확한 결과를 만드는 것은 아닙니다. 파일을 잘못 수정할 수도 있고, 필요 없는 코드를 추가할 수도 있고, 기존 구조를 깨뜨릴 수도 있습니다.

그래서 AI가 만든 결과를 바로 반영하는 것이 아니라, 먼저 어떤 파일이 바뀌었는지 확인해야 합니다. Git은 이 과정을 관리하기 위한 도구입니다.

Git을 사용하면 변경 전과 변경 후를 비교할 수 있고, 문제가 생겼을 때 이전 상태로 되돌릴 수 있습니다. 또 GitHub의 Pull Request를 이용하면 다른 사람에게 변경사항을 검토받을 수 있습니다.

Slide 3. Git과 GitHub 차이

화면 구성

- Git: 로컬 변경 이력 관리
- GitHub: 원격 저장소와 협업 플랫폼
- 이미지 후보: PC 안의 Git 저장소와 클라우드 GitHub 저장소 그림

발표 내용

Git과 GitHub는 비슷하게 들이지만 역할이 다릅니다.

Git은 내 컴퓨터에서 코드 변경 이력을 관리하는 도구입니다. 언제 어떤 파일을 수정했는지, 이전 버전과 무엇이 다른지 추적할 수 있습니다.

GitHub는 Git 저장소를 온라인에서 관리하고 협업할 수 있게 해주는 플랫폼입니다. 내 코드를 원격 저장소에 올리고, 다른 사람과 같이 작업하고, Pull Request를 통해 리뷰를 받을 수 있습니다.

정리하면 Git은 버전 관리 도구이고, GitHub는 Git을 기반으로 한 협업 서비스입니다.

Slide 4. Git 기본 흐름

화면 구성

- Working Directory
- Staging Area
- Commit
- Remote Repository
- 이미지 후보: 수정 -> add -> commit -> push 흐름도

발표 내용

Git의 기본 흐름은 네 단계로 볼 수 있습니다.

첫 번째는 파일 수정입니다. 두 번째는 수정한 파일을 staging area에 올리는 단계입니다. 이때 사용하는 명령어가 git add입니다.

세 번째는 변경사항을 하나의 기록으로 저장하는 commit입니다. 네 번째는 commit을 GitHub 같은 원격 저장소에 올리는 push입니다.

즉 가장 기본적인 명령어 흐름은 git status로 상태를 확인하고, git add .로 변경사항을 추가하고, git commit -m "메시지"로 기록을 만들고, git push로 원격 저장소에 올리는 것입니다.

Slide 5. Commit은 작업 저장 단위

화면 구성

- commit 1: HTML 구조 추가
- commit 2: 제품 페이지 수정
- commit 3: 이미지 경로 수정
- 이미지 후보: 시간순 commit 그래프

발표 내용

Commit은 단순 저장 버튼이 아니라, 작업 단위를 기록하는 것입니다.

예를 들어 제품 페이지를 만든다고 하면, 모든 작업을 한 번에 commit하는 것보다 기능별로 나누는 것이 좋습니다.

첫 번째 commit은 제품 페이지 구조 추가, 두 번째 commit은 제품 설명 추가, 세 번째 commit은 이미지 경로 수정처럼 나눌 수 있습니다.

이렇게 하면 나중에 어떤 작업에서 문제가 생겼는지 찾기 쉽고, 리뷰하는 사람도 변경사항을 이해하기 쉽습니다.

Slide 6. Branch는 독립적인 작업 공간

화면 구성

- main에서 feature branch가 갈라지는 그림
- 이미지 후보: main branch와 shj-update branch 흐름도

발표 내용

Branch는 main 코드와 분리해서 작업할 수 있는 독립적인 공간입니다.

main은 기준이 되는 안정 버전으로 두고, 실제 수정은 새로운 branch에서 진행합니다.

예를 들어 제품 페이지를 수정한다면 update-product-pages 같은 branch를 만들고 작업합니다. 작업이 끝나면 Pull Request를 통해 main에 합쳐달라고 요청합니다.

이 방식의 장점은 main을 깨끗하게 유지할 수 있다는 점입니다. 또 작업이 잘못되었을 때 branch만 삭제하면 되기 때문에 안전합니다.

Slide 7. Fork, origin, upstream

화면 구성

- 회사 원본 저장소
- 내 fork 저장소
- 내 로컬 PC
- 이미지 후보: upstream / origin 관계도

발표 내용

회사나 다른 사람의 저장소에서 작업할 때는 보통 Fork를 사용합니다.

Fork는 원본 저장소를 내 GitHub 계정으로 복사하는 것입니다. 내가 직접 회사 원본 저장소에 push하지 않고, 내 fork에서 작업한 뒤 Pull Request를 보내는 방식입니다.

여기서 origin은 내 fork 저장소를 의미하고, upstream은 회사 원본 저장소를 의미합니다.

예를 들어 회사 저장소가 TITeng-Korea에 있고, 내 계정이 tjdguswls01이라면, upstream은 회사 원본이고 origin은 내 fork입니다.

Slide 8. Pull Request란 무엇인가

화면 구성

- 내 branch에서 회사 main으로 PR 보내는 그림
- 이미지 후보: 실제 GitHub PR 화면 캡처

발표 내용

Pull Request는 내가 작업한 내용을 원본 저장소에 합쳐달라고 요청하는 과정입니다.

PR에는 어떤 파일이 변경되었는지, 어떤 commit이 포함되었는지, 충돌이 있는지 확인할 수 있습니다.

관리자나 팀원은 PR을 보고 코드 리뷰를 하거나, 수정 요청을 하거나, 문제가 없으면 merge할 수 있습니다.

AI가 만든 코드도 마찬가지입니다. AI가 수정한 결과를 바로 main에 반영하는 것이 아니라, PR을 통해 변경사항을 검토받는 것이 안전합니다.

Slide 9. VS Code에서 Git 사용

화면 구성

- VS Code Source Control 패널 캡처
- 변경 파일 목록
- commit message 입력창

발표 내용

VS Code에서는 Git을 터미널 명령어로만 사용하는 것이 아니라, 왼쪽 Source Control 패널에서도 사용할 수 있습니다.

파일을 수정하면 Source Control 탭에 변경된 파일이 표시됩니다. 파일을 클릭하면 이전 버전과 현재 버전의 차이를 볼 수 있습니다.

또 commit message를 입력하고 commit을 만들 수도 있습니다. Git 명령어가 익숙하지 않은 사람은 VS Code의 UI를 통해 Git 흐름을 먼저 이해하는 것도 좋습니다.

Slide 10. 터미널에서 Git 사용

화면 구성

- VS Code 터미널 캡처
- git status, git add, git commit, git push 명령어 표시

발표 내용

VS Code 안에서도 터미널을 열면 Git 명령어를 직접 사용할 수 있습니다.

가장 기본적인 명령어는 git status, git add ., git commit -m "메시지", git push origin branch-name입니다.

처음에는 어렵게 느껴질 수 있지만, 이 네 가지 흐름만 익혀도 기본적인 협업 작업은 가능합니다.

Slide 11. AI 도구와 Git을 같이 쓰는 방식

화면 구성

- Codex/Copilot이 코드 수정
- Git diff 확인
- commit
- PR 생성

- 이미지 후보: AI 개발 워크플로우 다이어그램

발표 내용

AI 도구를 사용할 때는 Git이 더 중요해집니다.

예를 들어 Codex나 Copilot에게 HTML 페이지를 수정해달라고 요청하면, AI가 여러 파일을 한 번에 바꿀 수 있습니다.

이때 바로 commit하지 말고 먼저 git diff 또는 VS Code diff 화면으로 변경 내용을 확인해야 합니다.

확인할 부분은 의도한 파일만 수정되었는지, 기존 링크나 구조가 깨지지 않았는지, 불필요한 코드가 추가되지 않았는지입니다.

검토가 끝난 뒤 commit하고 push하는 것이 안전합니다.

Slide 12. 실무에서 자주 생기는 실수

화면 구성

- main에서 바로 작업
- pull 안 하고 push
- commit message 부정확
- branch 잘못 선택
- 이미지 후보: 체크리스트 카드

발표 내용

Git을 처음 사용할 때 자주 생기는 실수가 있습니다.

첫 번째는 main에서 바로 작업하는 것입니다. 작은 개인 프로젝트에서는 가능하지만, 협업에서는 작업 branch를 만드는 것이 좋습니다.

두 번째는 원격 저장소의 최신 내용을 받지 않고 작업하는 것입니다. 다른 사람이 먼저 수정한 내용이 있으면 push가 거부될 수 있습니다.

세 번째는 commit message를 너무 대충 쓰는 것입니다. 나중에 기록을 보면 어떤 작업인지 알기 어렵습니다.

네 번째는 PR 방향을 잘못 잡는 것입니다. 내 branch에서 회사 main으로 요청해야 하는데, 반대로 잡으면 문제가 생길 수 있습니다.

Slide 13. 발표 예시: 제품 페이지 작업 흐름

화면 구성

- 제품 HTML 작성
- branch 생성
- commit
- push
- PR
- 이미지 후보: 실제 작업 흐름 캡처

발표 내용

이번 제품 페이지 작업을 예로 들면 흐름은 다음과 같습니다.

먼저 제품 소개 HTML 페이지를 만듭니다. 그 다음 shj-update 같은 branch에서 수정합니다. 수정이 끝나면 commit을 만들고 GitHub에 push합니다. 그리고 회사 원본 저장소로 Pull Request를 생성합니다.

이렇게 하면 작업자는 자유롭게 수정할 수 있고, 관리자는 PR 화면에서 변경사항을 검토할 수 있습니다.

Slide 14. 정리

화면 구성

- Git = 변경 이력 관리
- GitHub = 협업
- VS Code = 작업 환경
- AI = 코드 작성 보조
- 이미지 후보: 4개 키워드 요약 카드

발표 내용

정리하면 Git은 변경 이력을 관리하는 도구이고, GitHub는 협업을 위한 플랫폼입니다. VS Code는 Git과 GitHub를 편하게 사용할 수 있는 개발 환경입니다.

AI 도구는 코드 작성과 수정 속도를 높여주지만, 결과 검증은 여전히 개발자의 역할입니다.

따라서 AI 시대의 개발 흐름은 AI가 초안을 만들고, Git으로 변경사항을 확인하고, GitHub PR로 리뷰를 받고, 사람이 최종적으로 검토한 뒤 반영하는 구조입니다.

이상으로 AI 시대에 Git을 활용하는 방법에 대해 설명을 마치겠습니다.

3번 발표 시나리오 - AI Coding과 Antigravity IDE 사용

발표 파일명: topic/antigravity.html

발표 목표

이 발표는 Antigravity IDE를 깊게 실습하는 발표가 아니라, AI Coding이 기존 코드 자동완성에서 Agent 기반 개발 방식으로 어떻게 확장되고 있는지 설명하는 발표이다. Antigravity IDE는 Agent 중심 개발 환경을 설명하기 위한 사례로 사용한다.

전체 시간 구성

구간	시간
도입: AI Coding이란 무엇인가	5분
기존 AI 코딩 방식	7분
Agent 기반 개발 방식	10분
Antigravity IDE 개념 소개	12분
장점과 한계	4분
마무리	2분
합계	40분

Slide 1. 제목

화면 구성

- 제목: AI Coding과 Antigravity IDE 사용
- 부제: 코드 자동완성에서 Agent 기반 개발로
- 이미지 후보: AI Agent, IDE, Terminal, Browser가 연결된 그림

발표 내용

이번 발표에서는 AI Coding이 무엇인지, 그리고 Antigravity IDE를 통해 Agent 기반 개발 방식이 어떻게 변화하고 있는지 설명하겠습니다.

AI Coding은 단순히 코드를 자동완성하는 수준에서 시작했지만, 최근에는 사용자가 목표를 제시하면 AI가 계획을 세우고 파일을 수정하고 실행 결과까지 확인하는 방향으로 발전하고 있습니다.

Slide 2. AI Coding이란 무엇인가

화면 구성

- 코드 작성
- 코드 설명
- 오류 분석
- 리팩토링
- 테스트 작성
- 이미지 후보: AI Coding 기능 카드 5개

발표 내용

AI Coding은 개발 과정에서 AI를 활용하는 방식입니다.

예를 들어 코드를 작성하거나, 기존 코드를 설명받거나, 오류 원인을 분석하거나, 반복적인 코드를 리팩토링하는 작업에 AI를 사용할 수 있습니다.

예전에는 개발자가 코드를 처음부터 끝까지 직접 작성하고, 검색을 통해 문제를 해결하는 방식이 일반적이었습니다. 하지만 지금은 AI에게 초안을 만들게 하고, 개발자가 그 결과를 검토하고 수정하는 방식으로 바뀌고 있습니다.

Slide 3. 기존 AI 코딩 방식

화면 구성

- ChatGPT
- GitHub Copilot
- Codex
- Claude Code
- 이미지 후보: 각 도구의 화면 캡처

발표 내용

기존 AI 코딩 방식은 크게 세 가지로 볼 수 있습니다.

첫 번째는 ChatGPT처럼 질문하고 답변을 받는 방식입니다. 이 방식은 개념 설명이나 코드 예시를 받을 때 좋습니다.

두 번째는 GitHub Copilot처럼 IDE 안에서 자동완성이나 채팅으로 도움을 받는 방식입니다. 현재 열려 있는 파일이나 선택한 코드를 기준으로 수정하기 편합니다.

세 번째는 Codex나 Claude Code처럼 프로젝트 단위로 코드를 이해하고 수정하는 Agent형 도구입니다. 이 방식은 여러 파일을 함께 분석하거나, 터미널 명령과 연결해 작업하기 좋습니다.

Slide 4. AI Coding의 변화

화면 구성

- 1단계: 자동완성
- 2단계: Chat 기반 수정
- 3단계: Agent 기반 작업
- 이미지 후보: 단계별 발전 흐름도

발표 내용

AI Coding은 크게 세 단계로 발전하고 있다고 볼 수 있습니다.

첫 번째 단계는 자동완성입니다. 개발자가 코드를 쓰는 중간에 다음 코드를 추천해주는 방식입니다.

두 번째 단계는 Chat 기반 수정입니다. 개발자가 이 코드를 설명해줘 또는 이 함수를 수정해줘라고 요청하면 AI가 답변합니다.

세 번째 단계는 Agent 기반 작업입니다. 사용자가 목표를 주면 AI가 계획을 세우고, 필요한 파일을 찾고, 수정하고, 실행 결과를 확인하는 방식입니다.

Antigravity IDE는 이 세 번째 단계, 즉 Agent 기반 개발 흐름을 설명하기 좋은 예시입니다.

Slide 5. Agent 기반 개발 방식

화면 구성

- 목표 입력
- 계획 수립
- 파일 수정
- 실행/검증
- 결과 보고
- 이미지 후보: Agent workflow 다이어그램

발표 내용

Agent 기반 개발 방식은 사용자가 세부 명령을 하나씩 내리는 방식과 다릅니다.

사용자는 먼저 목표를 제시합니다. 예를 들어 제품 소개 페이지를 만들고, 각 제품별 링크를 연결해줘라고 요청합니다.

그러면 AI Agent는 필요한 파일을 찾고, 작업 계획을 만들고, 파일을 수정하고, 가능하다면 실행이나 검증까지 진행합니다.

개발자는 AI가 만든 결과를 검토하고, 잘못된 부분을 수정하도록 다시 지시합니다.

즉 개발자는 코드를 직접 작성하는 역할뿐 아니라, AI에게 작업을 지시하고 결과를 검토하는 역할까지 하게 됩니다.

Slide 6. 일반 IDE와 Agent 중심 IDE의 차이

화면 구성

- 일반 IDE: 사람이 파일 선택, 코드 작성, 실행
- Agent IDE: 사람이 목표 제시, Agent가 파일 탐색/수정/실행
- 이미지 후보: 기존 IDE vs Agent-first IDE 비교표

발표 내용

일반적인 IDE에서는 개발자가 직접 파일을 열고 코드를 수정하고 실행합니다. AI 기능이 붙어 있어도 대부분은 개발자가 현재 보고 있는 코드 주변에서 도움을 받는 형태입니다.

반면 Agent 중심 IDE에서는 AI가 더 적극적으로 작업합니다. 프로젝트 전체 구조를 보고, 필요한 파일을 찾고, 여러 파일을 동시에 수정하고, 터미널이나 브라우저와 연결해 결과를 확인할 수 있습니다.

이 차이가 앞으로 개발 도구가 바뀌는 핵심 포인트라고 볼 수 있습니다.

Slide 7. Antigravity IDE란 무엇인가

화면 구성

- Antigravity IDE 로고 또는 메인 화면
- 키워드: Agentic Development Platform
- 이미지 후보: Antigravity 공식 메인 화면 캡처

발표 내용

Antigravity IDE는 Google에서 소개한 Agent 중심 개발 플랫폼입니다.

일반적인 코드 편집기처럼 파일을 열고 수정할 수 있는 기능도 있지만, 핵심은 AI Agent가 개발 과정을 함께 수행한다는 점입니다.

사용자는 작업 목표를 제시하고, Agent는 작업 계획과 결과물을 만들어냅니다. 이 과정에서 코드 편집기, 터미널, 브라우저 같은 개발 도구가 하나의 흐름으로 연결됩니다.

Slide 8. Antigravity IDE의 구성 요소

화면 구성

- Editor
- Terminal
- Browser
- Agent Manager
- Artifacts
- 이미지 후보: 구성 요소 다이어그램

발표 내용

Antigravity IDE를 설명할 때는 몇 가지 구성 요소로 나누어 볼 수 있습니다.

첫 번째는 Editor입니다. 기존 IDE처럼 코드를 보고 수정하는 공간입니다.

두 번째는 Terminal입니다. 빌드, 테스트, 실행 명령을 처리하는 공간입니다.

세 번째는 Browser입니다. 웹 프로젝트의 결과를 확인하거나 화면을 검증하는 데 사용할 수 있습니다.

네 번째는 Agent입니다. Agent는 사용자의 목표를 받아 작업 계획을 세우고 실제 변경을 수행합니다.

마지막으로 Artifacts는 Agent가 만든 작업 계획, 결과 화면, 검증 내용 같은 산출물입니다. 이것을 통해 사용자는 AI가 무엇을 했는지 검토할 수 있습니다.

Slide 9. Antigravity IDE 사용 흐름 예시

화면 구성

- 목표 입력: 제품 소개 페이지 생성
- Agent 계획
- 파일 수정
- 결과 확인
- 이미지 후보: 단계별 화면 캡처 또는 목업

발표 내용

예를 들어 제품 소개 페이지를 만든다고 가정해보겠습니다.

사용자가 제품 10개에 대한 HTML 페이지를 만들고, 각 페이지를 제품 목록과 연결해줘라고 입력합니다.

Agent는 먼저 현재 프로젝트 구조를 확인합니다. 그 다음 필요한 HTML 파일을 찾고, 제품 페이지 구조를 설계합니다. 이후 파일을 수정하고, 링크가 맞는지 확인합니다.

사용자는 Agent가 만든 결과를 보고, 디자인이 마음에 들지 않으면 다시 수정 요청을 할 수 있습니다.

이 방식은 개발자가 모든 코드를 직접 입력하는 것보다 빠르게 초안을 만들 수 있다는 장점이 있습니다.

Slide 10. AI Agent 활용 시 좋은 점

화면 구성

- 반복 작업 감소
- 여러 파일 수정
- 코드 구조 이해
- 빠른 초안 제작

- 이미지 후보: 장점 카드 4개

발표 내용

AI Agent를 활용하면 반복적인 작업을 줄일 수 있습니다.

예를 들어 비슷한 구조의 HTML 파일을 여러 개 만들거나, 같은 형식의 README를 작성하거나, 여러 파일의 링크를 점검하는 작업에 유리합니다.

또 프로젝트 구조를 빠르게 파악하는 데도 도움이 됩니다. 새로운 코드베이스를 처음 봤을 때 파일 구조를 설명받거나, 어떤 파일부터 봐야 하는지 안내받을 수 있습니다.

특히 발표 자료나 제품 페이지처럼 비슷한 형식을 반복해서 만드는 작업에서는 AI Agent가 초안을 만드는 데 도움이 됩니다.

Slide 11. 한계와 주의점

화면 구성

- AI 결과 검토 필요
- 보안 주의
- 권한 관리
- 회사 코드 사용 주의
- 이미지 후보: 경고/체크리스트 카드

발표 내용

AI Agent가 강력해질수록 주의할 점도 커집니다.

첫 번째는 결과 검토입니다. AI가 만든 코드가 항상 정확한 것은 아니기 때문에, 개발자가 반드시 diff를 확인하고 테스트해야 합니다.

두 번째는 권한 문제입니다. Agent가 파일을 수정하거나 터미널 명령을 실행할 수 있다면, 잘못된 명령이 실행되지 않도록 주의해야 합니다.

세 번째는 보안입니다. 회사 내부 코드나 민감한 자료를 AI 도구에 입력할 때는 회사 정책을 확인해야 합니다.

AI Agent는 생산성을 높여주는 도구이지만, 최종 책임은 여전히 개발자에게 있습니다.

Slide 12. 펌웨어 개발자 관점

화면 구성

- C 코드
- 빌드 로그
- Makefile/CMake
- UART/I2C/SPI
- 이미지 후보: STM32 프로젝트 구조 예시

발표 내용

펌웨어 개발자 관점에서도 AI Coding은 활용할 수 있는 부분이 많습니다.

예를 들어 C 코드의 구조를 설명받거나, UART/I2C/SPI 드라이버 코드의 흐름을 분석하거나, 빌드 로그를 보고 원인을 찾는 작업에 사용할 수 있습니다.

또 Makefile이나 CMakeLists.txt처럼 처음 보면 복잡한 빌드 설정 파일을 분석하는 데도 도움이 됩니다.

다만 펌웨어는 하드웨어와 직접 연결되는 코드가 많기 때문에, AI가 만든 코드를 실제 보드에 적용하기 전에는 반드시 빌드와 동작 검증이 필요합니다.

Slide 13. 정리

화면 구성

- AI Coding = 초안 생성 + 검토
- Agent IDE = 목표 기반 작업
- Antigravity = Agent 중심 개발 사례
- 개발자 = 지시 + 검토 + 검증
- 이미지 후보: 핵심 요약 4개 카드

발표 내용

정리하면 AI Coding은 개발자가 코드를 전부 직접 작성하는 방식에서, AI가 초안을 만들고 개발자가 검토하는 방식으로 변화하고 있습니다.

Antigravity IDE는 이러한 변화 중에서도 Agent 중심 개발 환경을 보여주는 사례입니다.

하지만 AI Agent가 모든 것을 대신해주는 것은 아닙니다. 개발자는 목표를 명확히 제시하고, AI가 만든 결과를 검토하고, 실제 환경에서 검증해야 합니다.

따라서 앞으로의 개발자는 코드를 작성하는 능력뿐 아니라, AI에게 정확히 지시하고 결과를 검증하는 능력도 중요해질 것입니다.

4번 발표 시나리오 - 코덱스 플러그인 사용과 CLI 코딩

발표 파일명: topic/codex.html

발표 목표

이 발표는 Codex CLI를 중심으로, AI 코딩 도구가 IDE 안에서만 쓰이는 것이 아니라 터미널 기반 개발 흐름과도 연결될 수 있음을 설명하는 발표이다. 또 Codex App, Codex IDE Extension, Codex CLI의 차이를 정리하고, Copilot, Claude Code, Codex CLI를 비교한다.

전체 시간 구성

구간	시간
도입: CLI 코딩이란 무엇인가	5분
Codex 사용 방식 비교	10분
플러그인/확장 기능 사용 방향	7분
Copilot, Claude Code, Codex CLI 비교	10분
펌웨어 개발 관점 활용	6분
마무리	2분
합계	40분

Slide 1. 제목

화면 구성

- 제목: 코덱스 플러그인 사용과 CLI 코딩
- 부제: Copilot, Claude Code, Codex 비교
- 이미지 후보: 터미널, VS Code, AI Agent 아이콘

발표 내용

이번 발표에서는 Codex CLI를 중심으로 CLI 기반 AI 코딩 방식에 대해 설명하겠습니다.

또 Codex를 사용하는 여러 방식, 즉 Codex App, Codex IDE Extension, Codex CLI를 비교하고, Copilot, Claude Code, Codex 같은 AI 코딩 도구가 어떤 차이가 있는지 정리하겠습니다.

Slide 2. CLI란 무엇인가

화면 구성

- CLI = Command Line Interface
- PowerShell / Bash / VS Code Terminal
- 이미지 후보: VS Code 터미널 캡처

발표 내용

CLI는 Command Line Interface의 약자입니다.

쉽게 말하면 마우스로 버튼을 누르는 방식이 아니라, 터미널에서 명령어를 입력해서 프로그램을 실행하는 방식입니다.

예를 들어 Git을 사용할 때도 git status, git add, git commit, git push 같은 명령어를 입력합니다.

Codex CLI는 이 터미널 환경에서 AI 코딩 에이전트를 실행하는 방식입니다.

Slide 3. CLI 기반 AI 코딩이란

화면 구성

- 프로젝트 폴더
- Codex CLI
- 파일 분석
- 코드 수정
- 명령 실행
- 이미지 후보: 프로젝트 폴더와 터미널 연결 그림

발표 내용

CLI 기반 AI 코딩은 터미널에서 AI에게 작업을 요청하는 방식입니다.

예를 들어 프로젝트 폴더에서 Codex CLI를 실행하고, 이 프로젝트의 HTML 링크 구조를 점검해줘라고 요청할 수 있습니다.

펌웨어 프로젝트라면 빌드 로그를 보고 에러 원인을 분석해줘 또는 UART 수신 코드의 상태머신 문제를 찾아줘라고 요청할 수 있습니다.

즉 CLI 기반 AI 도구는 코드 작성뿐 아니라, 빌드, 로그, Git 작업과 연결하기 쉽다는 장점이 있습니다.

Slide 4. Codex 사용 방식 3가지

화면 구성

- Codex App
- Codex IDE Extension
- Codex CLI
- 이미지 후보: 세 방식 비교 카드

발표 내용

Codex는 하나의 방식으로만 사용하는 도구가 아닙니다.

첫 번째는 Codex App입니다. 별도의 앱에서 Codex 작업을 관리하는 방식입니다. 여러 작업을 나눠서 관리하기 좋습니다.

두 번째는 Codex IDE Extension입니다. VS Code 같은 IDE 안에서 Codex를 사용하는 방식입니다. 코드 파일을 보면서 수정하고 변경사항을 확인하기 편합니다.

세 번째는 Codex CLI입니다. 터미널에서 Codex를 실행하는 방식입니다. 빌드 명령, Git 명령, 로그 분석 같은 터미널 작업과 연결하기 좋습니다.

Slide 5. Codex App

화면 구성

- Codex App 화면 캡처
- 키워드: 작업 관리, 병렬 작업, diff 확인

- 이미지 후보: 앱 화면 캡처

발표 내용

Codex App은 별도의 앱에서 Codex 작업을 관리하는 방식입니다.

예를 들어 여러 개의 작업을 동시에 진행하거나, 각각의 작업을 thread 단위로 관리하고 싶을 때 적합합니다.

제품 페이지 작업으로 예를 들면, 하나는 제품 설명 보완, 하나는 디자인 수정, 하나는 링크 점검처럼 작업을 나누어 관리할 수 있습니다.

다만 발표 주제인 CLI 코딩과는 실행 위치가 다릅니다. Codex App은 GUI 기반이고, Codex CLI는 터미널 기반입니다.

Slide 6. Codex IDE Extension

화면 구성

- VS Code 안의 Codex 패널
- 파일 변경사항 preview
- 이미지 후보: VS Code Codex Extension 캡처

발표 내용

Codex IDE Extension은 VS Code 안에서 Codex를 사용하는 방식입니다.

코드를 보면서 바로 질문할 수 있고, 수정 결과를 IDE 안에서 확인하기 쉽습니다.

처음 사용하는 사람에게는 CLI보다 IDE Extension이 더 편할 수 있습니다. 왜냐하면 파일 구조와 변경사항을 눈으로 보면서 작업할 수 있기 때문입니다.

하지만 터미널에서 빌드하거나 Git 명령어와 연결하는 흐름은 CLI가 더 자연스러운 경우가 많습니다.

Slide 7. Codex CLI

화면 구성

- codex 명령어 실행 화면
- 프로젝트 폴더 기준 작업
- 이미지 후보: VS Code 터미널에서 Codex 실행 캡처

발표 내용

Codex CLI는 터미널에서 실행하는 Codex입니다.

사용자는 프로젝트 폴더에서 Codex를 실행하고 자연어로 작업을 요청합니다.

예를 들어 이 HTML 프로젝트의 product 페이지 링크가 깨진 곳을 찾아줘 또는 이 C 프로젝트의 빌드 에러 원인을 분석해줘라고 요청할 수 있습니다.

Codex CLI의 장점은 터미널 작업과 바로 연결된다는 점입니다. Git, 빌드, 테스트, 로그 확인과 같은 작업을 같은 흐름에서 다룰 수 있습니다.

Slide 8. Codex App, Extension, CLI 비교

화면 구성

- 비교표
- 항목: 실행 위치, 장점, 단점, 적합한 상황

발표 내용

세 방식을 비교하면 다음과 같습니다.

Codex App은 별도 앱에서 여러 작업을 관리하기 좋습니다. Codex IDE Extension은 VS Code 안에서 파일을 보면서 수정하기 좋습니다. Codex CLI는 터미널에서 빌드, 로그, Git 작업과 연결하기 좋습니다.

따라서 하나가 무조건 더 좋다고보다는 사용 상황에 따라 선택하는 것이 맞습니다.

처음 배우는 단계에서는 IDE Extension이 편하고, 터미널 기반 작업이 많은 환경에서는 CLI가 유리합니다.

Slide 9. 플러그인과 확장 기능 사용 방향

화면 구성

- Superpowers
- Image Gen
- 문서 작성
- 이미지 생성
- 코드 보조
- 이미지 후보: 기능별 카드

발표 내용

Codex나 AI 도구를 사용할 때는 코드 작성 기능만 보는 것이 아니라, 작업을 보조하는 확장 기능도 함께 볼 수 있습니다.

예를 들어 Superpowers 같은 확장 기능은 AI 작업 흐름을 보완하는 도구로 소개할 수 있습니다.

Image Gen은 코드 자체를 작성하는 기능은 아니지만, 발표 자료나 UI 목업, 설명용 이미지를 만들 때 활용할 수 있습니다.

즉 AI 도구는 코드 작성뿐 아니라 문서, 이미지, 발표자료 제작까지 확장될 수 있습니다.

다만 플러그인이나 확장 기능은 사용 환경에 따라 실제 제공 기능이 다를 수 있기 때문에, 발표 전에는 직접 실행 화면을 캡처해서 확인하는 것이 좋습니다.

Slide 10. Copilot, Claude Code, Codex CLI 비교

화면 구성

- Copilot
- Claude Code
- Codex CLI
- 비교표 이미지

발표 내용

AI 코딩 도구를 비교할 때는 실행 위치와 작업 방식이 중요합니다.

GitHub Copilot은 주로 IDE 안에서 사용하기 위한 AI 코딩 보조 도구입니다. 코드 자동완성, 채팅, 선택 영역 수정 같은 작업에 강점이 있습니다.

Claude Code는 터미널에서 실행되는 AI 코딩 도구로 볼 수 있습니다. 코드베이스를 이해하고, 파일을 수정하고, 명령을 실행하는 작업에 사용할 수 있습니다.

Codex CLI도 터미널 기반 코딩 에이전트입니다. 프로젝트 폴더를 기준으로 코드 분석, 수정, 명령 실행, Git 흐름과 연결하기 좋습니다.

Slide 11. Copilot의 특징

화면 구성

- VS Code 안의 Copilot Chat
- inline suggestion 예시
- 이미지 후보: Copilot 자동완성 화면

발표 내용

Copilot의 가장 큰 장점은 IDE 안에서 바로 사용할 수 있다는 점입니다.

코드를 작성하는 중간에 자동완성을 받을 수 있고, 선택한 코드에 대해 설명을 요청하거나 수정할 수 있습니다.

따라서 빠른 코드 작성이나 현재 파일 중심의 수정에는 Copilot이 편합니다.

하지만 터미널 중심 작업이나 빌드 로그 분석, 여러 파일을 다루는 프로젝트 단위 작업에서는 CLI형 도구가 더 적합할 수 있습니다.

Slide 12. Claude Code와 Codex CLI의 특징

화면 구성

- 터미널 화면 2개 비교
- Claude Code / Codex CLI
- 이미지 후보: 두 도구의 실행 화면 캡처

발표 내용

Claude Code와 Codex CLI는 모두 터미널 기반 AI 코딩 도구로 설명할 수 있습니다.

두 도구 모두 프로젝트 코드를 읽고, 파일을 수정하고, 명령 실행과 연결될 수 있습니다.

이런 도구는 특히 터미널을 많이 사용하는 개발자에게 유용합니다.

예를 들어 웹 개발에서는 테스트 실행이나 빌드 로그 분석에 사용할 수 있고, 펌웨어 개발에서는 컴파일 에러, 링커 에러, Makefile, CMake, UART 로그 분석 등에 활용할 수 있습니다.

Slide 13. 펌웨어 개발자 관점에서 CLI가 유용한 이유

화면 구성

- C 코드
- Makefile/CMake
- 빌드 로그
- Git diff
- 이미지 후보: STM32 또는 임베디드 프로젝트 폴더 구조

발표 내용

펌웨어 개발자는 터미널 기반 작업을 많이 합니다.

예를 들어 make, cmake, ninja, arm-none-eabi-gcc, openocd, STM32_Programmer_CLI 같은 도구를 사용할 수 있습니다.

이런 작업은 IDE 화면보다 터미널 로그가 중요할 때가 많습니다.

Codex CLI 같은 도구를 사용하면 빌드 로그를 붙여넣거나, 프로젝트 폴더를 기준으로 에러 원인을 분석하도록 요청할 수 있습니다.

또 Git diff를 확인하면서 AI가 수정한 코드가 적절한지 검토할 수 있습니다.

Slide 14. CLI 코딩의 안전한 사용 흐름

화면 구성

- AI 요청
- 파일 수정
- git diff
- build/test
- commit
- PR
- 이미지 후보: 안전한 AI 코딩 흐름도

발표 내용

CLI 기반 AI 코딩을 사용할 때는 안전한 흐름이 중요합니다.

먼저 AI에게 작업을 요청합니다. 그 다음 AI가 수정한 파일을 바로 믿지 않고 git diff로 변경사항을 확인합니다.

그 후 빌드나 테스트를 실행해서 문제가 없는지 확인합니다. 문제가 없으면 commit을 만들고, GitHub에 push한 뒤 Pull Request를 생성합니다.

이 흐름을 지키면 AI의 생산성을 활용하면서도 코드 품질을 관리할 수 있습니다.

Slide 15. 정리

화면 구성

- Codex App: 작업 관리
- Codex Extension: IDE 안에서 편함
- Codex CLI: 터미널과 연결
- Copilot: IDE 보조
- Claude Code: 터미널 Agent
- 이미지 후보: 최종 비교 요약 카드

발표 내용

정리하면 Codex는 App, IDE Extension, CLI 등 여러 방식으로 사용할 수 있습니다.

Codex App은 작업을 관리하기 좋고, Codex IDE Extension은 VS Code 안에서 코드 수정과 변경사항 확인이 편하며, Codex CLI는 터미널 기반 개발 흐름과 잘 맞습니다.

Copilot은 IDE 안에서 빠르게 코드를 작성하고 수정하는 데 유리하고, Claude Code와 Codex CLI는 터미널 기반 Agent 작업에 가깝습니다.

프웨어 개발자 관점에서는 코드 작성에는 IDE Extension이나 Copilot이 편하고, 빌드 로그, Git, 톨체인 작업에는 CLI 방식이 유용할 수 있습니다.

따라서 중요한 것은 특정 도구 하나만 고르는 것이 아니라, 작업 환경에 맞게 AI 도구를 선택하고 결과물을 반드시 검토하는 것입니다.

HTML 구성 요약

파일	구성
topic/github.html	슬라이드 14장 정도
topic/antigravity.html	슬라이드 13장 정도
topic/codex.html	슬라이드 15장 정도

제작 우선순위

- 1. topic/github.html
- 2. topic/codex.html
- 3. topic/antigravity.html

Git은 실제 작업 경험이 있어서 설명하기 쉽고, Codex는 현재 제품 페이지 작업 흐름과 직접 연결된다. Antigravity는 실제 사용 화면 캡처가 부족하면 개념 소개 중심으로 처리하면 된다.